

UNIVERSIDADE FEDERAL DE MINAS GERAIS

Instituto de Ciências Exatas

Departamento de Ciência da Computação

Curso de Graduação em Sistemas de Informação

Lucas Dolabella de Castro Lopes - 2023001549

Vanessa Nascimento Silva - 2023001816

Proteção Contra Desreferenciamento do Ponteiro no xv6

Belo Horizonte

2026

1. Introdução

Este trabalho consiste na implementação de proteção contra desreferenciamento de ponteiro nulo no sistema operacional xv6. No xv6 original, programas de usuário são carregados a partir do endereço virtual 0x0, de modo que desreferenciar um ponteiro nulo não gera nenhuma exceção, o sistema simplesmente lê os primeiros bytes do próprio código do programa. O objetivo deste trabalho é tornar a primeira página do espaço de endereçamento de todo processo inválida, de forma que qualquer acesso ao endereço 0x0 cause uma exceção de acesso inválido à memória e o processo seja encerrado pelo kernel.

A solução adotada foi reposicionar os programas de usuário para iniciar no endereço 0x1000 (segunda página), deixando a primeira página permanentemente não mapeada. Foram realizadas modificações no linker script dos programas de usuário, no Makefile, na função `exec()` do kernel e no tratador de falhas de página `vmfault()`. Um programa de teste null foi criado para demonstrar o comportamento correto.

2. Modificação 1 - Reposicionamento dos Programas de Usuário

2.1. Descrição

No xv6 original, o linker script `user/user.ld` define o endereço de carga de todos os programas de usuário como 0x0. Para que a primeira página fique permanentemente livre (não mapeada), é necessário mover o início dos programas para 0x1000. O programa `forktest` possui uma regra de linkagem própria no Makefile que também precisou ser atualizada.

2.1. Diff - user/user.ld

```
diff --git a/user/user.ld b/user/user.ld
index 3da93e0..db02ec0 100644
--- a/user/user.ld
+++ b/user/user.ld
@@ -2,7 +2,7 @@ OUTPUT_ARCH( "riscv" )

SECTIONS
{
- . = 0x0;
+ . = 0x1000;

.text : {
    *(.text .text.*)
}
```

2.2. Diff - Makefile

```
diff --git a/Makefile b/Makefile
index 39b2738..cdbf558 100644
--- a/Makefile
+++ b/Makefile
@@ -113,7 +113,7 @@ $U/usys.o : $U/usys.S
```

```

$U/_forktest: $U/forktest.o $(ULIB)
# forktest has less library code linked in - needs to be small
# in order to be able to max out the proc table.
- $(LD) $(LDFLAGS) -N -e main -Ttext 0 -o $U/_forktest $U/forktest.o $U/ulib.o
$U/usys.o
+ $(LD) $(LDFLAGS) -N -e main -Ttext 0x1000 -o $U/_forktest $U/forktest.o $U/ulib.o
$U/usys.o
$(OBJDUMP) -S $U/_forktest > $U/forktest.asm

mkfs/mkfs: mkfs/mkfs.c $K/fs.h $K/param.h
@@ -148,6 +148,7 @@ UPROGS=\
    $U/_dorphan\
    $U/_ltest\
    $U/_graph\
+   $U/_null\

fs.img: mkfs/mkfs README.md $(UPROGS)
mkfs/mkfs fs.img README.md $(UPROGS)

```

A alteração em `user.ld` desloca o segmento de texto de todos os programas de usuário para `0x1000`, deixando a primeira página (`0x0-0xFFFF`) nunca alocada e nunca mapeada. O `forktest` utiliza `-Ttext` diretamente na linha de linkagem em vez do `user.ld`, por isso precisou ser ajustado separadamente. O programa `_null` foi adicionado à lista de programas compilados para servir como teste.

3. Modificação 2 - Ajuste do Espaço de Endereçamento em `exec()`

3.1. Descrição

A função `exec()` em `kernel/exec.c` recebe duas modificações. A primeira inicializa a variável `sz`, que rastreia o tamanho atual do espaço de endereçamento durante o carregamento do ELF. No original, `sz` começa em 0, o que permite que segmentos sejam mapeados a partir da primeira página. Para garantir que a página nula jamais seja alocada, `sz` é inicializado em `PGSIZE`, forçando o loader a começar a partir da segunda página.

A segunda adiciona uma verificação explícita sobre o endereço virtual de cada segmento ELF (`ph.vaddr`). Sem essa guarda, um ELF malformado com `ph.vaddr = 0x0` passaria pelas verificações anteriores e chegaria à função `loadseg`, que acionaria `panic("loadseg: address should exist")` que é um kernel panic em vez de uma rejeição graciosa. Com a verificação, `exec` retorna -1 de forma controlada.

3.2. Diff - `kernel/exec.c`

```

diff --git a/kernel/exec.c b/kernel/exec.c
--- a/kernel/exec.c
+++ b/kernel/exec.c
@@ -28,7 +28,7 @@ kexec(char *path, char **argv)
{
    char *s, *last;
    int i, off;
-   uint64 argc, sz = 0, sp, ustack[MAXARG], stackbase;

```

```

+ uint64 argc, sz = PGSIZE, sp, ustack[MAXARG], stackbase;
+ struct elfhdr elf;
+ struct inode *ip;
+ struct proghdr ph;
@@ -66,6 +66,8 @@ kexec(char *path, char **argv)
+     if(ph.vaddr % PGSIZE != 0)
+         goto bad;
+     if(ph.vaddr < PGSIZE)
+         goto bad;
+     uint64 sz1;
+     if((sz1 = uvmmalloc(pagetable, sz, ph.vaddr + ph.memsz, flags2perm(ph.flags))) ==
0)
+         goto bad;

```

Estas duas alterações, combinadas com a mudança no linker script, garantem consistência em ambas as direções e em todas as camadas: o binário é linkado para iniciar em 0x1000; o kernel nunca aloca memória abaixo de PGSIZE durante o carregamento; e qualquer ELF que tente declarar um segmento na página nula é rejeitado de forma graciosa antes de causar um panic.

4. Modificação 3 - Tratamento de Referências Nulas no Kernel

4.1. Descrição

O xv6 neste projeto utiliza alocação de páginas sob demanda (demand paging). Quando um processo acessa uma página ainda não mapeada, o processador gera uma falha de página (page fault) e o kernel chama vmfault() para tentar resolver o acesso alocando a página. Para interceptar acessos ao endereço nulo, foi adicionada uma verificação em vmfault(): se o endereço da falha for menor que PGSIZE, a função retorna 0 imediatamente, sinalizando ao tratador de interrupções que o acesso é inválido. O tratador então chama setkilled(p), encerrando o processo.

4.2. Diff - kernel/vm.c

```

diff --git a/kernel/vm.c b/kernel/vm.c
index 28f4248..4526553 100644
--- a/kernel/vm.c
+++ b/kernel/vm.c
@@ -458,6 +458,8 @@ vmfault(pagetable_t pagetable, uint64 va, int read)
+     if (va >= p->sz)
+         return 0;
+     va = PGROUNDDOWN(va);
+     if (va < PGSIZE)
+         return 0;
+     if(ismapped(pagetable, va)) {
+         return 0;
+     }

```

Não foi necessário alterar nenhum outro código de validação de ponteiros no kernel. O caminho de validação existente nas chamadas de sistema (fetchaddr → copyin → walkaddr) já rejeita corretamente o endereço 0x0, pois walkaddr retorna 0 para qualquer página não

mapeada, fazendo copyin falhar e a syscall retornar erro.

5. Programa de Teste - user/null.c

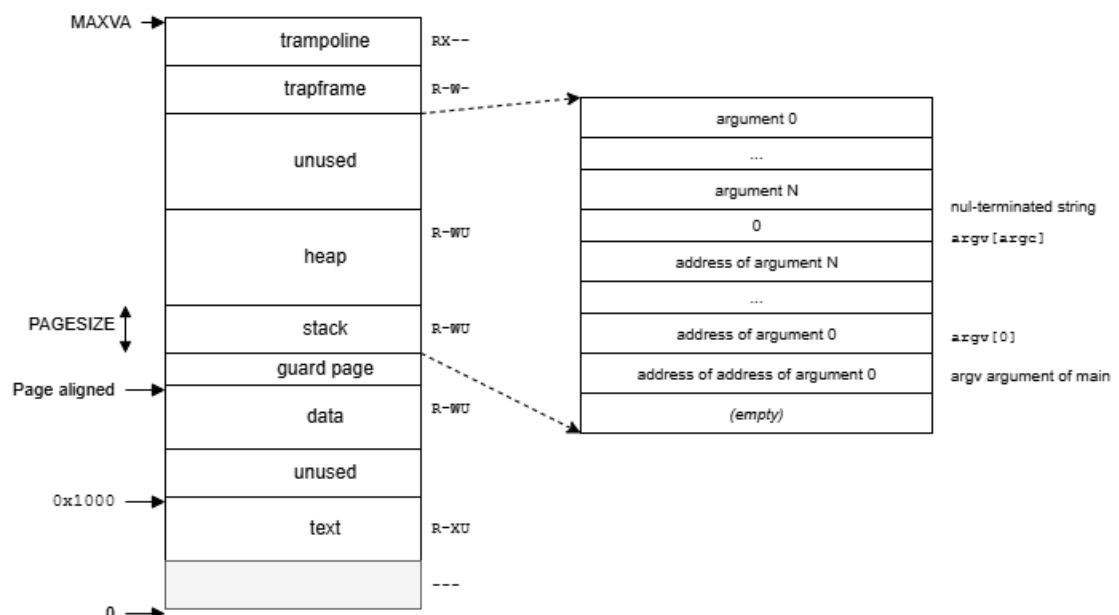
5.1. Diff - user/null.c (arquivo novo)

```
diff --git a/user/null.c b/user/null.c
new file mode 100644
index 0000000..8343db1
--- /dev/null
+++ b/user/null.c
@@ -0,0 +1,12 @@
+#include "kernel/types.h"
+#include "user/user.h"
+
+int
+main(void)
+{
+    printf("null: dereferencing null pointer...\n");
+    int *p = 0;
+    printf("value at null: %d\n", *p);
+    printf("null: should not reach here\n");
+    exit(0);
+}
\ No newline at end of file
```

O programa atribui o endereço 0 a um ponteiro e tenta lê-lo. A segunda e terceira chamadas a printf nunca devem ser executadas; se o processo for encerrado na desreferência, a proteção está funcionando corretamente.

6. Espaço de Endereçamento do Processo

O espaço de endereçamento do processo após as modificações é idêntico ao da Figura 3.4 do livro xv6, com uma alteração na parte inferior: a primeira página, antes ocupada pelo início do segmento de texto, passa a ser uma página nula inválida e permanentemente não



7. Testes e Resultados

Os testes foram realizados executando o xv6 dentro do QEMU com make CPUS=1 qemu.

7.1. Teste de desreferenciamento de ponteiro nulo

Ao executar o programa null no shell do xv6, obteve-se:

```
$ null
null: dereferencing null pointer...
usertrap(): unexpected scause 0xd pid=3
          sepc=0x1014 stval=0x0
$
```

O resultado confirma o comportamento esperado:

- A primeira linha foi impressa, confirmando que o programa iniciou corretamente em 0x1000;
- scause 0xd (= 13) indica uma falha de página de leitura (load page fault) que é o tipo correto de exceção para acesso a página não mapeada;
- stval=0x0 é o endereço da falha, confirmando que foi exatamente o acesso ao ponteiro nulo que causou a exceção;
- sepc=0x1014 é o endereço da instrução que causou a falha, dentro do segmento de texto que começa em 0x1000;
- A mensagem "null: should not reach here" nunca foi impressa, confirmando que o processo foi encerrado no momento da desreferência;
- O prompt \$ retornou imediatamente, confirmando que o kernel sobreviveu.

7.2. Teste de ausência de vazamento de memória

O programa null foi executado três vezes consecutivas. O sistema permaneceu responsivo após cada execução, sem pânico ou travamento:

```
$ null
usertrap(): unexpected scause 0xd pid=3  stval=0x0
$ null
usertrap(): unexpected scause 0xd pid=4  stval=0x0
$ null
usertrap(): unexpected scause 0xd pid=5  stval=0x0
$ echo ainda funcionando
ainda funcionando
$
```

O incremento dos PIDs (3, 4, 5) e a resposta correta ao echo confirmam que cada processo foi devidamente encerrado e sua memória liberada pelo kernel.

7.3. Teste de regressão: programas normais

Os programas echo e ls foram executados após os testes de ponteiro nulo e

funcionaram corretamente, confirmando que as modificações não introduziram regressões no comportamento normal do sistema.

8. Conclusão

A proteção contra desreferenciamento de ponteiro nulo foi implementada com sucesso no xv6. As modificações realizadas foram: reposicionamento de todos os programas de usuário para o endereço 0x1000 via alteração no linker script user/user.ld e no Makefile; ajuste da variável sz em exec() para iniciar em PGSIZE, garantindo que a página nula nunca seja alocada pelo kernel; e adição de uma verificação em vmfault() para rejeitar qualquer acesso a endereços abaixo de PGSIZE, resultando no encerramento imediato do processo infrator.

Os testes confirmaram que processos que desreferenciarem um ponteiro nulo são corretamente identificados pelo kernel como acessos inválidos à memória e encerrados, que não há vazamento de memória após o encerramento, e que programas normais continuam funcionando sem alteração de comportamento.